A request is not a fill

Study date: 22 October 2026. Task L002b: event ordering, fees and missing fills. Sidiq review status: Not yet reviewed. Synthetic teaching examples only.

Research question and hypothesis

What may change at a decision, and what must wait for a confirmed fill? The pre-check hypothesis is that a request changes neither cash nor shares, a fee is charged only with an accepted buy, and absent fill evidence leaves a requested order unresolved. Invalid event order must fail explicitly. This follows the October 20 L002a hand timeline; no October 21 lesson package was found, so no study or reason is inferred for that date. Today defines one bounded contract.

The data are six hand-authored synthetic examples, with ordered teaching events rather than a historical market period. There is no fit/evaluation split or parameter search. Metrics are exact state/status equality, rejection of ten invalid traces, and detection of five corrupted tables. Two runs of a separate per-share Decimal calculation must produce identical JSON. C++ remains the default for the next component; Python here verifies the contract before that implementation.

Beginner explanation and terminology

An observation tells us something already available. A decision uses that information to request a trade. An order is that request; a fill is evidence that the trade executed. They are distinct events. A later price printed in a table does not mean it was known at the earlier decision.

A fee schedule tells us the amount our model would charge; a fee charge is the actual cash event. Knowing the schedule early does not mean paying it early. A missing fill price means this fixture has no execution evidence. Unresolved means we cannot conclude that execution happened by the fixture horizon. Rejected_by_model means a supplied proposal fails our ledger's rules; it is not an observed exchange rejection. No_order means the decision made no request at all.

The scope is one asset, one order, whole money units and whole shares. No debt, shorts, fractions, partial fills, reservation, retry or competing orders are modelled. Starting cash and shares are in 0..10000. Positive prices and buy quantity are at most 10000; final shares cannot exceed 10000. Today quantity is fixed at two and fee at three. These caps and units are educational choices.

Which information is known when?

Event	Available information	Allowed accounting change
Observation 0	Price 50 becomes available; cash 800, shares 4	None
Decision 1	Observation 50, threshold 50, quantity 2, fee schedule 3	Request only; keep cash 800 and shares 4
Resolution 2	Supplied fill price 52, or explicit missingness	Fill if valid and affordable; otherwise preserve state

Require 0 <= observation_available < decision < resolution. Equal indices are rejected because we have not defined ordering inside one teaching event. This strict policy is deliberately narrower than a real timestamp system; do not import it as a universal market rule. Invalid chronology invalidates the trace. It must not be relabelled as an ordinary unfilled or unaffordable order.

Derive the contract slowly

The decision rule requests n=2 shares if the known observed price is at most 50; otherwise n=0. It is a fixed teaching rule without evidence of profitability. The decision must not inspect the later fill price. No cash reservation occurs; affordability is assessed at the supplied fill.

For a valid accepted buy, let C be cash, q shares, p fill price in money per share, and f the fee. Notional is n*p. Debit D=n*p+f, new cash C_new=C-D, and new shares q_new=q+n. All operations have explicit units: shares times money per share gives money. Subtracting shares directly from cash would be a unit error even if a programming language allowed it.

Start with C=800, q=4, n=2, p=52 and f=3. Debit is 2*52+3=107. After the fill, cash is 693 and shares are 6. At the common mark 52, pre-trade wealth is 800+4*52=1008; post-trade wealth is 693+6*52=1005. The fee reduces same-mark wealth by 3. Relative to the original mark 50 and wealth 1000, the old four shares gained 8 before the fee. The result is not a five-unit trading profit from the newly bought shares, and it is not a market-performance claim.

Missingness, rejection and boundaries

Case	Input change	Outcome	Final cash/shares	Fee charged
Supplied fill	Fill 52	filled	693/6	3
Missing fill	Fill absent	unresolved	800/4	0
Exact cash	Cash 107; fill 52	filled	0/6	3
Insufficient cash	Cash 106; fill 52	rejected_by_model	106/4	0
No request	Observed price 51	no_order	800/4	0
Position limit	Starting shares 9999	rejected_by_model	800/9999	0

The table resets initial state for each case. It is not six consecutive orders. The fixed fee is charged exactly once with an accepted buy. If 106 cannot fund debit 107, neither cash nor shares changes; deducting just the fee would violate this policy. Exact cash 107 is accepted, leaving zero. No_request is determined from the observed price, even if a later fill proposal is supplied.

For missing fill evidence we store explicit null, not zero. Zero is an invalid supplied price; null is missingness. Filling at the earlier observed price would invent an event. Unresolved also does not imply cancellation or permanent non-execution. The fixture ends without resolving that uncertainty. A real system would require subsequent events and an explicit lifecycle.

Implementation and reproducible checks

The saved delayed_order_contract.md defines the policy. delayed_order_contract_cases.json contains the six hand expectations, including both decision and final state. verify_event_contract.py is reference verification tooling; it is not the new C++ trading component or a general execution engine. From the repository root run: `.venv/bin/python scripts/verify_event_contract.py`.

The note multiplies quantity by price. The reference debits one share at a time using Decimal, then subtracts the fee from a temporary candidate balance. It updates the returned state only when cash and position checks pass. Hand-written expected status, decision state, final state and fee must match exactly. Both methods share the policy and may share a wrong assumption.

Ten invalid traces cover observation equality/future availability, resolution equality/earliness, negative event indices, zero/negative/Boolean fill prices, fractional cash and negative cash. They fail with the intended domain or ordering diagnostic. Five copied tables deliberately claim a premature debit, fee on missing fill, invented old-price fill, fee on rejection, or trade without request. Every corruption is detected with a state/status mismatch.

All six correct cases pass and both result replays agree byte for byte. The real Python -O invocation is refused so disabled assertions cannot silently skip checks. Initial lint found a long scope string; an initial format check then requested reformatting. Both were corrected. The final reference passes lint and format checks. No arithmetic/runtime failure was observed.

Original October 22 make check passed 608 Python tests, Ruff and 59 file formatting checks, strict mypy on 21 package files, 66 savings checks, 30 parser checks and all existing C++ fixtures/oracles, including the five-program fresh accounting reproduction. The reference was then checked separately. On October 23, make event-contract joins the tracked checks and the current suite is revalidated. Dependency consistency passes without a new dependency. Standard Make binaries are reused; temporary drivers/reproduction build fresh. No full clean environment recreation is claimed.

Costs, failure cases and limitations

The fee of three is chosen for teaching, not a broker tariff. Missing-fill handling is our explicit evidence policy. The SEC explains that a last-traded price need not equal the execution price; that motivates separating observation from fill, but does not prescribe our caps or fees. The model omits spread formation, impact, liquidity, financing, tax and settlement. It does not predict whether a submitted real order would fill or how a broker would reserve funds.

The equality rule uses event indices, not exchange timestamp precision, time zones or latency. There is no vector, generic queue, multiple-asset replay or portfolio reservation system today. The domain checker intentionally rejects a malformed supplied fill even in a no-order trace. A future interface may choose a different policy, but must document and test the change.

These six cases and corrupt tables verify a reference against a written contract. They do not prove the future C++ decision avoids later data. L002d must exercise that component with changed future prices and a deliberately leaking control. No measured coverage percentage, investment return or empirical alpha is reported. Q004a/Q005b licence and provenance gates remain in force.

Interview explanation and next step

I distinguish the information used to request a trade from the later evidence that it executed. I preserve cash and shares at the decision, apply the actual supplied fill and fee together, and leave a missing fill unresolved. I test equality boundaries and false accounting claims. Passing a reference worksheet establishes a contract baseline; implementation invariance remains separate work. This is a repository learning outcome, not a claim of personal mastery.

Next is L002c: one bounded C++ delayed-order component using this contract and the completed L001 accounting primitive. Introduce only necessary syntax, then compare its trace with the hand expectations. L002d future-price invariance, L002e adverse fills and L002f reproduction remain separate lessons. C003b CLI, C004 returns/vectors, C005 statistics and strategy prerequisites remain unfinished. Do not jump to the historical trend or pairs calendar windows.

Integration on October 23 preserves the original study date and results. All 26 original input/artifact/QA records verify before integration. The published fixture path and Make contract check are the only verifier integration changes. Six cases, all invalid/corrupt controls and two replays are revalidated. L002a was integrated separately earlier today. This uses the second of two commit slots; today's new C++ lesson stays pending. Personal review remains Not yet reviewed. No input blocks the next bounded synthetic step.

Primary sources consulted October 22

QuantConnect, Timeslices: its time frontier illustrates limiting information to what is available at simulated algorithm time. Our strict event-index policy is a separate teaching choice. https://www.quantconnect.com/docs/v2/writing-algorithms/key-concepts/time-modeling/timeslices

SEC Investor.gov, Types of Orders: last-traded and execution prices can differ. The page is context for distinguishing an observation from execution, not evidence about our synthetic fills. https://www.investor.gov/introduction-investing/investing-basics/how-stock-markets-work/types-orders

Clang, UndefinedBehaviorSanitizer: compile-time instrumentation checks selected executed paths in the existing C++ buy tests. It does not establish financial correctness or full coverage. https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

Check your understanding

1. At decision event 1, which fields change in the 800-cash, four-share example?

2. Why do initial cash 107 and 106 produce different outcomes at fill 52 with fee 3?

3. Why must missing fill evidence not become a fill at the observed price 50?

4. Do passing hand/reference cases prove the future C++ decision avoids future prices?

Suggested answers

1. Only the request record changes to two shares. Cash stays 800 and shares stay four. The fee schedule is known, but no execution or fee charge has occurred at the decision.

2. Required debit is 2*52+3=107. Equality is sufficient: cash 107 leaves zero and shares become six. Cash 106 cannot fund the debit; rejection preserves 106 and four shares with no fee.

3. It would invent execution evidence and a price. Leave the order unresolved and preserve state; missing evidence proves neither cancellation nor that execution can never happen later.

4. No. Today checks a contract reference. The implementation still needs future-price mutations and a deliberately leaking control, alongside the accounting and timing tests.